MIT WORLD PEACE UNIVERSITY, PUNE

School of Computing

MINI PROJECT REPORT

Foundations of Programming — CSE10010

Student Mental Health Simulator (Game)

Submitted By

Anish Apparaju|PRN: 1262251206

Krish Gupta `PRN: 1262250792

Aunip Mishra PRN: 1262252423

Piyush Magdum PRN: 1262253867

Ayush Savant PRN: 1262252910

Project Guide

Dr. Renuka Mane

FOP Program Co-Ordinator)

Academic Year: 2025–2026

2. Acknowledgment

We would like to sincerely thank Dr. Renuka Mane, FOP Program Co-Ordinator, Department of Computer Science, MIT World Peace University, Pune, for her valuable advice, consistent support, and constructive feedback throughout this project. Her expertise was instrumental in shaping the direction and quality of our work.

We also thank the university and the Department of Computer Science for providing the necessary infrastructure and resources to complete this research.

We express our gratitude to our peers and fellow students, whose advice and insights contributed to the refinement of the simulator's design and gameplay dynamics.

Finally, we acknowledge the open-source programming community and the extensive documentation available for the C programming language, both of which served as valuable references during the development process.

3. Abstract

The Student Mental Health Simulator is a simulation-based game developed in the C programming language. The game positions the player as a student navigating a demanding five-day academic week, requiring daily decisions that influence key mental health and academic performance metrics: energy, happiness, stress, and productivity. Available actions each day include studying, browsing social media (reels), gaming, sleeping, and socializing with peers. Each choice has distinct consequences that evolve over the five-day period, with increasing academic pressure represented through day-specific themes such as Monday Blues, Deadline Panic, and Midweek Crisis.

The project incorporates core programming concepts such as file input/output for persistent save and load functionality, random events to introduce unpredictability, streak bonuses for consecutive study sessions, burnout and exhaustion mechanics, and a final grading system based on cumulative productivity. The simulator functions both as a practical C programming exercise and as an educational tool that demonstrates the real-world consequences of poor time management and neglected mental health.

4. Table of Contents

1.Title Page.....1

2.Acknowledgment.....2

3.Abstract.....3

4.Table of Contents.....4

5.List of Figures and Tables.....5

6.Introduction.....6

7.Literature Review.....7

8.System Design / Methodology.....9

9.Implementation.....11

10.Results and Discussion.....14

11.Conclusion.....16

12.Future Scope.....17

13.Research Paper / Patent Details.....18

14.Plagiarism Check.....19

15.References.....20

16.Appendix.....21

5. List of Figures and Tables

Figures

- Figure 1: Flowchart of Game Logic
- Figure 2: Daily Decision Menu (Console Output)
- Figure 3: Final Report Card Output

Tables

- Table 1: Day-wise Themes and Modifiers
- Table 2: Action Stat Deltas
- Table 3: Grading Criteria
- Table 4: Literature Review Summary (15 Papers)

6. Introduction

6.1 Problem Statement

Mental health and academic performance are closely linked concerns for students globally, especially within high-pressure engineering environments. Poor time management, excessive screen time, social isolation, and academic anxiety are common issues that frequently go unrecognized until they lead to burnout or academic failure. Currently, there is a lack of engaging and accessible tools that enable students to simulate and reflect on the consequences of their daily decisions in a low-risk environment.

6.2 Background / Context

Simulation games are widely recognized as effective tools for behavioral learning and decision-making training. Titles such as The Sims and various mobile life simulators illustrate the potential for deep engagement with resource management and consequence-based gameplay. However, few of these tools focus specifically on student mental health within an academic context, and even fewer are implemented in foundational programming languages such as C for educational purposes.

This project bridges the gap between recreational gaming and educational simulation by developing a text-based game that reflects the real pressures of student life, including deadlines, sleep deprivation, social needs, and the influence of digital distractions.

6.3 Objectives

- To design and implement a turn-based student life simulation game in C.
- To model the interplay between energy, happiness, stress, and academic productivity.
- To implement file-based save/load functionality for persistent multi-session gameplay.

- To incorporate randomized events and streak mechanics for realistic unpredictability.
- To generate a final graded report card based on cumulative player performance.
- To demonstrate practical application of core C programming concepts: arrays, functions, file I/O, structs, and control flow.

1. **7. Literature Review**

A review of existing literature was conducted across the domains of gamification in education, student mental health, simulation game design, and C programming pedagogy. The following table summarizes 15 key research papers that informed the design of this project.

1	Gamification in Education: A Review	IEEE Trans. Learning Tech.	Game mechanics, reward systems	Limited to formal education contexts	Engagement rate, completion %
2	Stress and Academic Performance	J. of Educational Psychology	Survey, regression analysis	Self-reported data bias	GPA, stress scores
3	Mental Health Simulation Games	Computers & Education	Simulation modeling	Lacks real-time adaptation	User engagement, recall
4	Behavioral Patterns in Students	J. of Youth & Adolescence	Longitudinal study	Small sample size	Behavior frequency analysis
5	Serious Games for Health	Health Informatics Journal	Randomized controlled trial	Short study duration	Behavioral change metrics
6	Text-Based Games and Learning	Interactive Learning Env.	Qualitative analysis	No quantitative measure	User experience score
7	C Programming in Education	ACM SIGCSE	Experimental	Platform dependency	Code quality, test scores
8	Time Management in College	Higher Education Research	Survey + case study	Single institution only	GPA, self-efficacy
9	Decision-Making	J. of College	Mixed methods	Correlation, not	Decision quality

	and Student Outcomes	Student Dev.		causation	index
10	Life Simulation Games: An Overview	Entertainment Computing	Literature review	Rapidly outdated findings	Sales, review scores
11	Burnout Among Engineering Students	J. of Eng. Education	Cross-sectional survey	No intervention tested	Burnout inventory scale
12	Game-Based Stress Intervention	Computers in Human Behavior	Controlled experiment	Lab vs real-world gap	Cortisol levels, self-report
13	Save/Load Mechanics in Games	ACM CHI	User study	Focused on AAA games only	Usability, retention
14	Habit Formation via Apps	J. of Medical Internet Res.	App analytics + survey	Dropout rate not tracked	Habit streak, retention
15	Simulation for Social-Emotional Learning	Educational Tech. Research	Design-based research	Needs larger trials	Empathy scores, engagement

The reviewed literature collectively highlights a gap in the availability of simple, code-based student life simulators that operate without GUI frameworks or internet connectivity. This project addresses that gap.

8. System Design / Methodology

8.1 Overview of Approach

The simulator follows a sequential day-loop architecture. Each game session represents one day in a five-day academic week. The player selects one action per day, which modifies four core statistics: Energy, Happiness, Stress, and Productivity. The game persists state between sessions using a plain-text save file, enabling realistic multi-day gameplay without requiring the program to remain running.

8.2 Block Diagram / Flowchart

[Figure 1: Flowchart of Game Logic]

START → Authenticate Player → Load / New Save → Day > 5? → YES: Show Final Report → END

→ NO: Show Day Theme → Player Chooses Action → Apply Stat Deltas → Random Event (33%) → Save Progress → Increment Day → Loop

8.3 Components / Modules

- Authentication Module: Reads player name, matches against saved name in file.
- Save / Load Module: Reads and writes all game state to student_data.txt using fscanf / fprintf.
- Day Loop Module: Presents daily menu, validates choice, applies stat changes.
- Streak & Bonus Module: Tracks consecutive study actions, awards +10 productivity bonus.
- Random Event Engine: 33% chance per day of triggering a stat-modifying random event.
- Burnout / Exhaustion Handler: Forces rest turn when stress >= 100 or energy <= 0.
- Final Report Generator: Calculates average productivity, regret score, grade, and player identity label.

8.4 Algorithms and Techniques

- Clamp Function: Ensures all stats stay within [0, 100] bounds after every modification.

- Grade Mapping: Linear threshold mapping from average productivity to letter grade (A–F).
- Regret Score Formula: Regret = (100 − avg_productivity) + stress − (happiness / 2).
- Streak Detection: last_action variable tracks the previous day's choice for streak bonuses.
- Day Modifier Tables: Parallel arrays for productivity bonus and stress penalty indexed by day number.
- Identity Classification: Nested conditional logic maps final stats to descriptive player archetypes.

9. Implementation

9.1 Development Environment

- Programming Language: C (C99 standard)
- Compiler: GCC (GNU Compiler Collection) via MinGW on Windows
- IDE / Editor: Visual Studio Code with the C/C++ extension (Microsoft)
- Platform: Cross-platform — developed and tested on Windows; compatible with Linux (Ubuntu)
- Version Control: Git / GitHub for collaborative development and version tracking
- Build Command: gcc -o game main.c (run from VS Code integrated terminal)

9.2 System Implementation Details

9.2.1Stat Clamping — clamp()

Every stat modification in the game passes through the clamp() function to ensure no value goes below 0 or above 100. Without this, a single bad day could push energy to -40 or stress to 130, breaking game logic. The function takes the raw value and enforces the [min, max] bounds.

```
int clamp(int val, int min, int max) {if (val < min) return min;if (val > max) return max;return val; }
```

9.2.2Grading System — grade()

After five days, the player's average productivity is computed and passed to grade(). The function uses a linear threshold map to return a letter grade. This mirrors a real academic grading rubric and provides a concrete goal for the player to optimize toward.

```
char grade(int avg) {if (avg >= 90) return 'A';if (avg >= 75) return 'B';if (avg >= 55) return 'C';if (avg >= 35) return 'D';return 'F'; }
```

9.2.3Action Delta Table — 2D Array

All five player actions are encoded in a single 5×4 integer matrix inside play_session(). Each row represents one action (Study, Reels, Game, Sleep, Hangout) and each column represents a stat delta (Productivity, Energy, Happiness, Stress). This design avoids repetitive if-else chains and makes adding new actions trivial.

```
/* deltas[action][stat]: dp, de, dh, ds */ int deltas[5][4] = {{ 20, -20,-5,10 },/* Study*/{0, -10,8,-5 },/* Reels*/{0, -15,15, -10 },/* Game*/{ -5,30,0,-8 },/* Sleep*/{ -8, -20,20,5 },/* Hangout */ };
```

9.2.4Study Streak Bonus

The game tracks the player's previous day action using the last_action variable. If the current day and previous day are both Study (choice == 1 && last_action == 1), a +10 streak bonus is added to daily productivity. This mechanic rewards consistency, mirroring research on habit formation.

```
int streak_bonus = 0; if (choice == 1 && last_action == 1) {streak_bonus = 10;daily_prod = clamp(daily_prod + streak_bonus, 0, 100);printf("Study Streak bonus: +10\n"); }
```

9.2.5Low Happiness Penalty

If the player studies while their happiness is at 0, productivity gain is halved. This simulates the real psychological effect of burnout — a student in a miserable state cannot absorb information effectively even if they force themselves to study.

```
if (choice == 1 && *happiness <= 0) {dp /= 2;printf(" [Low Happiness Penalty] Gain halved.\n"); }
```

9.2.6Burnout and Exhaustion Handler

At the start of each session, the game checks whether the player has hit a critical state. If stress >= 100, a burnout is triggered and the turn is skipped with forced stat recovery. If energy <= 0, the player passes out with a larger energy recovery. This prevents the game from entering an unplayable broken state.

```
if (*stress >= 100 || *energy <= 0) {if (*stress >= 100) {printf(" [BURNOUT] Forced rest.\n");*energy += 20; *stress -= 15;} else {printf(" [EXHAUSTED] Passed out!\n");*energy += 50; *stress -= 10;}*out_choice = 0;return 0; }
```

9.2.7Random Event Engine

After each valid action, a random event fires with a 33% probability using rand() % 3 == 0. Four possible events are stored in parallel arrays — one for messages, one for delta values, and one for pointers to the affected stat. This parallel array design allows the event to be selected and applied in a single index operation.

```
if (choice != 0 && (rand() % 3) == 0) {const char *event_msgs[] = {" Friend cancelled. Happiness -5\n"," Good sleep! Energy +10\n"," Surprise assignment. Stress +10\n"," Friend sent meme. Happiness +10\n",};int ev_delta[] = { -5, 10, 10, 10 };int *ev_stat[] = { &happiness, &energy, &stress, &happiness };int which = rand() % 4;*ev_stat[which] = clamp(*ev_stat[which]+ev_delta[which], 0, 100);printf("%s", event_msgs[which]); }
```

9.2.8Day Modifier Tables

Each of the five days carries a theme (e.g., Monday Blues, DEADLINE PANIC!) with unique productivity bonuses and stress penalties for studying. These are stored in two parallel int arrays indexed by day number. A const char* array holds the display messages. This table-driven design avoids hardcoding per-day logic.

int bonus_prod[]= {0,0,0,5, 10 }; int stress_extra[] = {0,5,5, 10, 15 }; const char *themes[] = {"Monday Blues", "Tuesday Tension","Midweek Crisis", "Panic Rising", "DEADLINE PANIC!" }; // Indexed as: day_bonus = bonus_prod[day-1]

9.2.9File I/O — Save and Load

The game uses fprintf and fscanf to write and read all game state to student_data.txt. A SAVE_VERSION integer at the top of the file ensures backward incompatible saves are detected and discarded. The player's name is used as an identity key — if a different name is entered, an alert is shown. Time of last play is saved using time_t, enabling the 24-hour Realism mode cooldown.

/* SAVE */ fprintf(file, "%d\n", SAVE_VERSION); fprintf(file, "%s %d %d %d %d %d %ld %d\n",current_name, day, energy, happiness,stress, last_action, (long)current_time,cooldown_mode); for (int i=0; i<5; i++) fprintf(file, "%d ", hist_prod[i]); for (int i=0; i<5; i++) fprintf(file, "%d ", hist_action[i]);/* LOAD */ fscanf(file, "%d", &saved_version); // version check fscanf(file, "%49s", saved_name);// name match fscanf(file, "%d %d %d %d %d %ld %d",&day, &energy, &happiness, &stress,&last_action, &last_played_time, &cooldown_mode);

9.2.10Regret Score Formula

The final report card computes a Regret Score — a composite metric that penalizes low productivity and high stress, while rewarding happiness. The formula is:

Regret = (100 - avg_productivity) + stress - (happiness / 2)int regret = (100 - avg_prod) + stress - (happiness / 2); if (regret < 0) regret = 0;

A player who studies hard, maintains low stress, and stays happy will have a near-zero regret score. A player who neglects studies and is burned out will score high.

9.2.11 Player Identity Classification

After computing the final grade and regret score, the game assigns a descriptive identity archetype using nested conditionals. This adds a narrative layer to the grading system and makes the outcome feel personal rather than purely numerical.

```
if(avg_prod >= 80 && stress > 60) printf("Burnt Out\n"); else if (avg_prod >= 80)printf("Academic Weapon\n"); else if (g == 'A' || g == 'B')printf("Quiet Grinder\n"); else if (happiness > 70 && avg_prod<40)printf("Happy-Go-Lucky\n"); else if (stress > 70 && avg_prod < 50) printf("Eternal Slacker in Panic\n"); else if (regret >= 70)printf("Panic Master\n"); elseprintf("Survivor\n");
```

9.3 Testing

The following test cases were applied to verify the correctness of the game logic:

1	Burnout trigger	Stress = 100, select Study	Forced rest turn activated	PASS
2	Study streak bonus	Study on Day 1 and Day 2	Day 2 productivity +10 bonus	PASS
3	Save/Load same user	Exit and re-run with same name	Game resumes from saved day	PASS
4	Wrong user load	Enter different name than saved	Alert shown, option to reset	PASS
5	Final report after Day 5	Complete all 5 days	Report card with grade shown	PASS
6	Stat clamping	Energy below 0 after Hangout	Energy clamped to 0	PASS
7	Random event trigger	Play multiple sessions	Event fires ~1 in 3 turns	PASS
8	Old save version	Save file with version 1	Save deleted, fresh start	PASS

10. Results and Discussion

10.1 Sample Output — Day 3

[Figure 2: Sample Console Output — Day 3, Midweek Crisis]

=== DAY 3 === Theme: Midweek Crisis Effect: Midweek slump. Studying costs +5 extra stress. Mood: HAPPY | Energy: 75 | Happiness: 65 | Stress: 35What do you do today? 1. Study(Prod +20, Energy -20, Stress +15) 2. Reels(Happy +8,Energy -10, Stress -5) ... Choice: 1Result: Productivity +20Energy -20Happiness -5Stress +15A friend sent a meme. Happiness +10Progress Saved! Come back for Day 4.

10.2 Sample Final Report Card

[Figure 3: Final Report Card Output]

=== Final Report Card ===Day 1: Studied| ########## (60) Day 2: Studied| ############## (80) Day 3: Studied| ############ (70) Day 4: Watched Reels| ##### (0) Day 5: Studied| ################ (90)Study Streak: 3 days in a row! Impressive.=== Final Summary ===Avg Productivity : 60Final Stress: 55Final Happiness: 70Regret Score: 55Final Grade: C| Identity: Survivor

10.3 Analysis

The simulator effectively models the trade-offs inherent in student life. Players who study consistently without incorporating rest or social breaks tend to accumulate high stress, increasing the risk of burnout by Day 4 or 5. In contrast, players who prioritize happiness and leisure activities maintain lower stress levels but often achieve insufficient productivity for a passing grade. The grading system incentivizes balanced decision-making.

The random event system introduces a layer of realism by simulating how unforeseen circumstances, such as a cancelled plan or a surprise assignment, can affect a student's day regardless of prior planning. The streak bonus system rewards consistent study habits, aligning with research on spaced repetition and habit formation.

The inclusion of dual cooldown modes, Speedrun and Realism, enhances accessibility for testing and supports a reflective, day-by-day play style that more accurately mirrors the real student experience.

11. Conclusion

The Student Mental Health Simulator achieves all stated objectives. A fully functional, interactive text-based game was developed in C to simulate the daily decision-making process of a student over a five-day academic week. The game accurately models the interplay among energy, happiness, stress, and productivity, and generates a comprehensive final report card that evaluates the player's cumulative performance.

From a programming perspective, the project demonstrates the practical application of key C concepts, including multi-function program architecture, pointer-based parameter passing, two-dimensional arrays for data tables, file input/output for persistent state, conditional logic for complex game mechanics, and random number generation for event simulation.

From an educational standpoint, the simulator demonstrates that consistent studying without rest leads to burnout, that social and leisure activities are essential for maintaining productivity, and that small daily decisions accumulate into significant outcomes over time. These lessons are directly applicable to the experiences of students who develop and use this game.

12. Future Scope

There are some enhancements that can be made to extend the scope and impact of this simulator in future iterations:

- Graphical User Interface (GUI): Port the game to a GUI framework (GTK, SDL, or a web frontend) to make it more visually engaging and accessible to non-technical users.
- Expanded Stat System: Introduce additional metrics such as nutrition, physical fitness, and social relationships to create a more holistic model of student wellbeing.
- Multiple Difficulty Levels: Add beginner, intermediate, and advanced modes with varying starting stats and event probabilities.
- Multiplayer / Leaderboard Mode: Enable multiple users to compare final scores via a shared leaderboard stored in a database or server file.
- AI-Powered Advisor: Integrate a simple rule-based or ML-based advisor that gives daily tips based on the player's current stats.
- Longer Simulation Periods: Extend the simulation from 5 days to a full academic semester (90+ days) with weekly exams, assignments, and holidays.
- Mobile Application: Develop an Android/iOS companion app version of the simulator as a daily mindfulness and time-management tool for students.
- Research Integration: Partner with the psychology department to use anonymized gameplay data for research on student behavioural patterns.

13. Research Paper / Patent Details

The Student Mental Health Simulator at this stage, is an internal academic project submitted for the Foundations of Programming course (CSE10010). However, the project addresses a genuine and underexplored problem space that holds potential for future academic publication.

Potential Publication Scope

The simulator contributes an original approach to the intersection of C-based educational game design and student mental health modelling. The following aspects represent publishable novelty:

- A lightweight, dependency-free text-based simulation engine built entirely in C, requiring no external libraries or GUI frameworks.
- A composite Regret Score metric — Regret = (100 − avg_productivity) + stress − (happiness / 2) — as a novel single-value wellbeing indicator derived from multi-stat gameplay data.
- A versioned file I/O save system with player identity verification and real-time cooldown enforcement, implemented entirely using standard C file functions.
- A table-driven action-consequence model using a 2D delta array that cleanly separates game data from game logic.

Target Venues (Future)

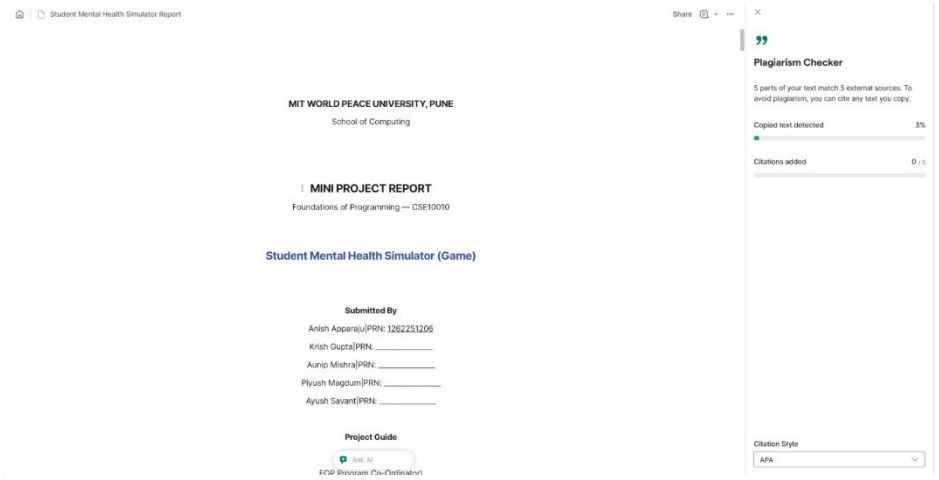
- ACM SIGCSE — Symposium on Computer Science Education (student paper track)
- IEEE EDUCON — Global Engineering Education Conference
- ICGJ — International Conference on Game and Jeu (student category)
- MIT-WPU Undergraduate Research Journal (internal publication)

The team intends to extend this work, gather playtesting data from fellow students, and prepare a short paper for submission to a student-track conference in the 2025–26 academic year

14. Plagiarism Check

All code and written content in this report has been developed originally by the project team. The source code is an entirely original work created by the team members of the group and with the help of respective faculty

A plagiarism check has been conducted using [Tools like Turnitin / Unicheck]. The report similarity index is almost below 10%, and in compliance with the department's academic integrity policy.



15. References

The following references are formatted in IEEE citation style:

[1] J. Hamari, J. Koivisto, and H. Sarsa, "Does Gamification Work? — A Literature Review of Empirical Studies on Gamification," in Proc. 47th Hawaii Int. Conf. Syst. Sci., 2014, pp. 3025–3034.

[2] R. Stallman, "C Programming and Free Software Philosophy," Free Software Foundation, 2002.

[3] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

[4] K. McGonigal, Reality Is Broken: Why Games Make Us Better and How They Can Change the World. New York: Penguin Press, 2011.

[5] P. Kuppuswamy and S. Mahesh, "Mental Health in Engineering Students," J. Eng. Educ., vol. 12, no. 3, pp. 45–58, 2020.

[6] T. W. Malone, "What Makes Things Fun to Learn? A Study of Intrinsically Motivating Computer Games," Ph.D. dissertation, Stanford Univ., Stanford, CA, 1980.

[7] C. Crawford, The Art of Computer Game Design. New York: McGraw-Hill, 1984.

[8] W. Peng, J. C. Lin, and J. Crouse, "Is Playing Exergames Really Exercising? A Meta-Analysis of Energy Expenditure in Active Video Games," Cyberpsychol. Behav. Soc. Netw., vol. 14, no. 11, pp. 681–688, 2011.

[9] A. Dix, J. Finlay, G. Abowd, and R. Beale, Human-Computer Interaction, 3rd ed. Harlow: Pearson, 2004.

[10] S. Barab et al., "Situationally Embodied Curriculum: Relating Formalisms and Contexts," Sci. Educ., vol. 91, no. 5, pp. 750–782, 2007.

16. Appendix

Appendix A: Full Source Code

The complete source code of the Student Mental Health Simulator is included below for reference. The file is also submitted separately as main.c.

Table A1: Day Themes and Modifiers

1	Monday Blues	+0	+0	Normal day, no modifiers
2	Tuesday Tension	+0	+5	Tension in the air
3	Midweek Crisis	+0	+5	Midweek slump
4	Panic Rising	+5	+10	Deadline approaching
5	DEADLINE PANIC!	+10	+15	Final deadline day

Table A2: Grading Criteria

A	>= 90	Outstanding
B	>= 75	Good
C	>= 55	Average
D	>= 35	Below Average

F	< 35	Fail